

Two Endpoints Built, One Webhook Neutralised, Zero Requests Sent

The documented payment API is implemented. The documented webhook authentication is not trustworthy, so the adapter stopped believing webhooks and started asking the provider instead. No request has ever been made to iPayMoney from this repository, and nothing below is offered as evidence of how the provider behaves.

Branch **claude/create-app-repository-1cxsih**

HEAD **c4ff2e6**

Unit tests **478 passed**

Browser tests **31 passed**

Mutations killed **24 / 26**

Real sandbox calls **0**

READ THIS BEFORE ANYTHING ELSE

Sandbox credentials do not exist, so no sandbox behaviour has been verified. The milestone asked for a sandbox integration; what could be built without credentials was built, and what needs credentials is written, gated and skipped. Sixteen tests in `ipaymoney.integration.test.ts` are skipped for exactly that reason, and one always-running test asserts the gate so the skip is visible in the output rather than buried in a config file.

Everything green below was proved against an **injected transport**. It proves the adapter is a faithful translation of the documentation *as we read it*. Only a real sandbox run can prove the documentation is true.

A

Commit, branch and what was touched

Three commits on `claude/create-app-repository-1cxsih`, none merged to `main`. The milestone began from `a37a21c`, the Phase 3 gate report.

COMMIT	SUBJECT	FILES	+	/	-
76d664a	iPayMoney: implement the two documented payment endpoints	9	1573	/	83
936cfc3	iPayMoney webhooks: check the notification, then ask the provider	5	729	/	70
c4ff2e6	Fix a racing e2e wait, and close three dangling references	4	110	/	7

Every file changed, and why

FILE	Δ	WHY
packages/core/src/payments/ipaymoney.ts	+681 -60	The adapter. <code>createCharge</code> , <code>getCharge</code> and <code>verifyWebhook</code> implemented; <code>refund</code> and <code>getRefund</code> still throw
packages/core/src/payments/ipaymoney.test.ts	+611 new	44 local tests over an injected <code>fetch</code> : configuration, request shape, conclusion classification, status mapping, webhook behaviour, derived event id, unsupported operations
packages/core/src/payments/ipaymoney-replay.test.ts	+266 new	5 database-backed tests. Proves five deliveries of one event grant one entitlement, through the real domain and the real unique index
packages/core/src/payments/ipaymoney.integration.test.ts	+262 -20	The real-sandbox suite. Gated on credentials <i>and</i> on <code>IPAYMONEY_ENVIRONMENT=san</code> <code>dbbox</code> ; every scenario the documented test numbers offer
packages/core/src/payments/provider.ts	+28 -1	<code>ChargeStatus.amount</code> becomes optional; <code>statesChargeAmount</code> added to the interface; <code>VerifiedEvent</code> becomes a discriminated union
packages/core/src/payments/mock.ts	+9	Declares <code>statesChargeAmount =</code> <code>true</code> . The mock does state amounts, and the contrast is the point
packages/core/src/payments/refund-outcome.ts	+24 -5	<code>ProviderTransportError</code> generalised from the refund path so charges classify transport failures by the same table
packages/core/src/payments/payments.test.ts	+39 -2	Contract tests updated for the optional amount and the new capability flag
docs/providers/ipaymoney/README.md	+81 -6	The webhook decision, the derived event id, and the assumptions register

FILE	Δ	WHY
docs/providers/ipaymoney/support-questions.md	+261 new	K1-K18, the message ready to send, ordered by what each answer blocks
docs/providers/ipaymoney/live-activation-checklist.md	+68 new	Prepared, not submitted
apps/web/e2e/delivery.spec.ts	+17 -1	A racing wait that CI caught. See section G

WHAT WAS DELIBERATELY NOT TOUCHED

OTP and authentication, consent, the checkout and late-payment rules, ledger posting, settlement, payout logic, and every unrelated product or content path. The refund state machine, the refund execution ordering and the `in_doubt` rules from the approved Phase 3 design are unchanged — no approved business rule was altered, so nothing in this milestone needed to stop and ask under rule 11.

What the adapter does, and what it refuses to do

METHOD	STATE	AGAINST
<code>createCharge()</code>	IMPLEMENTED	POST <code>/api/v1/payments</code>
<code>getCharge()</code>	IMPLEMENTED	GET <code>/api/v1/payments/{reference}</code>
<code>verifyWebhook()</code>	IMPLEMENTED	Header check, then a lookup. See section C
<code>refund()</code>	THROWS	Nothing. iPayMoney documents no customer-refund operation at any endpoint
<code>getRefund()</code>	THROWS	Declared and throwing rather than absent — see below
<code>createPayout()</code> · <code>getPayout()</code>	ABSENT	The documented <i>reversement</i> is a dashboard withdrawal of the merchant's own balance, not a disbursement API

No customer-refund API was invented, anywhere. There is no endpoint, method, request body, response shape, status value, webhook, sandbox scenario, fee or processing time for one in the documentation, so there is none in the code. The word « remboursement » appears throughout the document and means **reversement** every time — the merchant withdrawing their own holdings, with a 50 000 FCFA floor and a three-day rule. Building a customer refund on it would return the merchant's money to the merchant while a buyer waits.

`getRefund()` is declared and throws, while `createPayout()` is simply absent, and the asymmetry is deliberate. Callers branch on `supportsRefundQuery(provider)`, which reads whether the method exists. Omitting it would assert on iPayMoney's behalf that they have no refund-query capability, and that assertion makes every ambiguous refund a manual reconciliation *by design*. That is the operational reality today, but it should be true because iPayMoney said so — question K2 — not because this file decided it.

What `createCharge` refuses rather than guesses

Three fields iPayMoney requires have no home in `ChargeInput`. They are read from `metadata` and the call is **refused** when they are absent:

FIELD	SOURCE	WHY IT IS NOT DEFAULTED
msisdn	<code>customer.phone</code> , else <code>metadata.msisdn</code>	The payer's number is not derivable from an order
country	<code>metadata.country</code>	Documented only as "le code du pays de la transaction". Whether that is the buyer's country or the merchant's is question K17 , and Afrinext's sellers and buyers can be in different countries
customer_name	<code>metadata.customerName</code>	Listed with no optionality marked, and the documented 400 says "Missing params"
Ipay-Payment-Type	<code>channel</code> — <code>mobile</code> or <code>card</code>	No documented default exists

A currency other than XOF is refused outright: no minor-unit convention is documented for any other currency, and this is the codebase where **XOF has zero decimals** is a load-bearing fact.

A REAL INTEGRATION GAP THIS MILESTONE FOUND AND DID NOT PAPER OVER

`initiatePayment` does not yet pass the buyer's phone number or country to the provider, so it **cannot currently create an iPayMoney charge** — the adapter would refuse it, correctly. `ipaymoney-replay.test.ts` therefore inserts its payment row directly rather than routing through `initiatePayment` , and says so in the file. Wiring the buyer's details through the checkout path is real work that belongs to whichever milestone turns iPayMoney on, and it is listed in section L.

The three decisions this milestone had to make

Each one is a place the documentation stops and the code cannot. Each is recorded here with what was chosen, what it costs, and what would change it.

1 · `ChargeStatus.amount` — made optional, and `iPayMoney` states none

Afrinext's webhook boundary cross-checks the confirmed amount against the order's amount. `iPayMoney` supplies **no amount in either channel**: not in the webhook payload, and not in the payment-status response.

Three options existed. Two were refused:

OPTION	VERDICT	WHY
Copy our own order amount into the <code>ChargeStatus</code>	REFUSED	This is the failure the milestone brief named explicitly. The cross-check would then compare the order's amount against the order's amount and pass every time — a check that cannot fail is worse than no check, because it reads like one
Keep <code>amount</code> required and send a zero or a placeholder	REFUSED	A zero is a number. Downstream code cannot tell "the provider said nothing" from "the provider said nothing was charged"
Make <code>amount</code> optional and declare the capability	CHOSEN	Absence is representable, and <code>statesChargeAmount</code> makes it a fact a caller can branch on and a test can assert

```
// provider.ts
readonly amount?: Money | undefined; // ABSENT when the provider states none

// PaymentProvider
readonly statesChargeAmount: boolean; // mock: true · ipaymoney: false
```

The cost, stated plainly: for `iPayMoney` the amount cross-check *cannot run*. The charged amount is not independently verifiable from the provider at all. That is question **K8**, and it is a Settlement blocker (S3) rather than something the adapter can solve.

The real-sandbox suite contains a test named *CONFIRMS OR REFUTES that no amount is returned*. If `iPayMoney` does return one, that test fails loudly and `statesChargeAmount` becomes `true` on evidence.

2 · Missing `providerEventId` — derived, not invented

Afrinext's replay defence is `UNIQUE (provider, provider_event_id) on payment_events`. `iPayMoney` sends **no event id** and **retries five times** on any non-2xx, so duplicates are not a hazard to guard against — they are the documented behaviour.

```
ipaymoneyEventId(reference, confirmedStatus) = `${reference}:${confirmedStatus}`
```

INGREDIENT	WHY IT IS IN
reference	iPayMoney's own identifier for the payment, so two different payments cannot collide whatever else is equal
status	So a success and a failure for the <i>same</i> payment do not collapse into one another. Keying on the reference alone would silently discard a later, truthful event as a duplicate of an earlier, different one
the confirmed status	Never the claimed one. A claimed value would let anyone reaching the endpoint mint fresh ids at will, and each fresh id is a row and a lookup
no body hash	Deliberately absent. A retry that re-serialises the payload is the <i>same fact</i> ; a byte-level id would differ between retries and defeat the deduplication it exists for. The id names the fact, not the delivery

Proved against a real database, not asserted about a string: `ipaymoney-replay.test.ts` delivers one event five times through `applyProviderEvent` and the real domain, and asserts **one** entitlement, **one** event row, and a clean commerce reconciliation.

If iPayMoney does send an id we have not seen — question **K9** — theirs replaces ours and the derivation is deleted.

3 · The webhook signature — the ambiguity was isolated, then neutralised

THE SECURITY FINDING

The documentation describes webhook authentication three incompatible ways: the prose names a header `x-ipaymoney-secret` containing "une signature" (L332); the example shows `secret-hash` whose value is the secret key itself, `sk_...` (L338); and the setup instructions say to paste the API secret into a dashboard field called "Secret Hash".

Read together, that is a **shared secret echoed in a header** — not a signature over the request body. It does not cover the payload. Anyone who obtains the secret from a log, a proxy or a mis-scoped error report can forge any event, and the secret is transmitted on every single webhook.

The brief said: *if the documentation cannot establish the correct cryptographic verification algorithm, isolate the ambiguity and stop before implementing an unsafe verification scheme*. The ambiguity was isolated and put to you as three options. **You chose A.**

OPTION		CONSEQUENCE
A · Treat the notification as a trigger, then ask the provider	CHOSEN	Verification depends on iPayMoney's API being reachable, and costs one extra request per notification
B · Implement the header check alone and trust it	—	A leaked secret becomes a forged payment
C · Leave <code>verifyWebhook</code> throwing until K7 is answered	—	No payment can ever be confirmed, and the answer may take weeks

As implemented, `verifyWebhook` :

1 Checks the header in constant time

Both documented names are accepted, because the documentation contradicts itself and guessing wrong would reject every real webhook. Accepting either is not a weakening — the *value* still has to match, and the value is the whole check. The comparison is `timingSafeEqual`, with the length compared first because it throws on a length mismatch and the length of a secret is not the secret.

2 Reads exactly one field out of the body

The payment reference, and nothing else. Not the status, not the msisdn, not the external reference.

3 Asks iPayMoney directly

`GET /api/v1/payments/{reference}` over our own authenticated outbound connection. The returned event carries the **confirmed** status; the claim never reaches the caller.

4 Records what was claimed alongside what was confirmed

`raw.claimed` is the notification's own words, screened for secrets and truncated. It is what makes a forgery attempt visible after the fact rather than invisible.

Forging a notification therefore buys nothing. It can make Afrinext ask a question; it cannot answer one. This holds even if the secret leaks — which the documented scheme, on its own, does not.

What it costs, stated plainly: verification now depends on iPayMoney's API being reachable. When the lookup fails, nothing is confirmed, the route answers non-2xx, and iPayMoney retries — the event is not lost, and an unconfirmable notification is never treated as a confirmation. It does **not** fix the amount: the status endpoint states none either.

Question K7 stays open. If iPayMoney confirms a real body signature, the lookup becomes an optimisation rather than the safety property, and the method simplifies on evidence.

#	DECISION	REASONING
D1	IPAYMONEY_ENVIRONMENT has no default	<code>Ipay-Target-Environment</code> decides whether a request moves real money. A missing value must never fall back to <code>live</code> — so it falls back to nothing, and <code>isConfigured</code> is false until somebody says <code>sandbox</code> or <code>live</code> out loud
D2	The webhook is never believed	Section C.3. The confirmed status comes from our own outbound request, not from the notification
D3	Constant-time secret comparison	<code>timingSafeEqual</code> , length checked first. A byte-by-byte compare leaks the secret one character at a time to anyone who can time responses
D4	Credentials are checked before the request is built, not inside the try	This was a real bug an existing test caught. A missing API key thrown inside the transport catch would be reclassified as stage <code>unknown</code> — "the request may have reached the provider" — when the truth is the exact opposite: no request was built, so nothing could have happened. It is the same mistake in the same direction the whole refund phase exists to prevent, one layer down: a configuration error becoming an ambiguous payment
D5	Nothing attacker-influenced is stored unscreened	<code>raw.claimed</code> passes through <code>screenDetail</code> , which redacts bearer tokens, API keys and JWTs and truncates to 500 characters
D6	No credential, key or webhook secret is in source control	Every value is read from the environment. The Live-activation checklist is a checklist, not a store — the business and personal identifiers it lists belong in the dashboard
D7	An undocumented status is refused, not mapped	Section E
D8	Only XOF is accepted	No minor-unit convention is documented for any other currency, and this is a codebase where a wrong exponent is a factor of 100

Error classification — the Phase 3 rule, applied to charges

The approved correction — **unknown must never become failed** — was written for refunds. Charges now go through the same table, because the reasoning does not change when the money moves the other way.

WHAT HAPPENED	STAGE	BECOMES
DNS failure, connection refused, host unreachable, TLS certificate rejected, invalid URL	not_transmitted	A DEFINITE FAILURE — the bytes provably never left
Timeout after transmission, connection reset, broken pipe, aborted request	unknown	UNKNOWN — never failed
HTTP 5xx	transmitted	UNKNOWN — the request arrived; nothing documents what a server error means for the transaction (K14)
HTTP 4xx from <code>POST /payments</code>	—	A DEFINITE REFUSAL — the documented 4xx cases are named refusals and no payment was created
HTTP 200 with no reference	transmitted	UNKNOWN — the payment may exist and cannot be identified
Missing credentials	not_transmitted	A CONFIGURATION ERROR — no request was built (D4)

An HTTP status never decides whether the financial operation happened. The only shapes that may become definite are the ones that prove non-transmission, and a documented business refusal.

Status mapping — two entries, and everything else refused

```
IPAYMONEY_STATUS_MAP = {  
    succeeded: "succeeded",  
    failed:    "failed",  
}
```

Those are the only two status values that appear anywhere in the documentation (L183, L246, L364). Anything else throws `IPayMoneyUnknownStatusError` naming the exact string received.

A SCENARIO IS NOT A STATUS

The sandbox names five **scenarios** — `success`, `error`, `insufficient_fund`, `declined`, `pending` — reached through ten test numbers. The documentation never states which `status` string a declined payment carries. Mapping `declined` to `failed` would be a guess wearing the clothes of a fact.

Refusing is the mechanism, not an inconvenience. The first real sandbox run against a declined test number will throw with the exact value in the message — and that message is the evidence needed to complete the map. A permissive default would convert that evidence into a silent assumption, and a payment status is not a thing to be silently assumed.

The real-sandbox suite is written to harvest exactly this: it drives every non-pending scenario and reports the value it received. Question **K12** asks `iPayMoney` for the complete enumeration in parallel, because a sandbox may not exhibit every value a production account can.

THE HONEST ANSWER TO "WAS THE SANDBOX TESTED?"

No. There are no sandbox credentials in this environment, so no request has ever been made to iPayMoney from this repository. Nothing in this report may be quoted as evidence of how the provider behaves.

Two suites, and the difference between them is the whole point

FILE	TRANSPORT	PROVES	RUNS
ipaymoney.test.ts	injected <code>fetch</code>	Our translation of the documentation is faithful <i>to how we read it</i>	ALWAYS · 44 tests
ipaymoney-replay.test.ts	injected <code>fetch</code> , real database	Five deliveries grant one entitlement, through the real domain	ALWAYS · 5 tests
ipaymoney.integration.test.ts	real network	The documentation is true	SKIPPED · 16 of 17

A green local suite says the adapter is self-consistent. Only the third file can say iPayMoney agrees.

The gate, and why it checks two things

```
const hasSandbox =
  typeof baseUrl === "string" && baseUrl !== "" &&
  typeof apiKey === "string" && apiKey !== "" &&
  environment === "sandbox";          // checked, not assumed
```

These tests **create payments**. Running them against `live` would create real ones, so the gate requires the environment to say `sandbox` explicitly: a missing or mistyped value skips rather than proceeds.

One test in the file **always** runs and asserts the gate's own value, so "skipped" and "passed" are distinguishable from the test output without anybody reading a config file.

What is waiting for credentials

TEST	WHAT IT WOULD ESTABLISH
creates a payment and receives a reference	A1, A5, A9 — that the request shape is accepted at all
answers a status query for a reference it gave us	The lookup round-trip
CONFIRMS OR REFUTES that no amount is returned	K8. If an amount comes back, <code>statesChargeAmount</code> is wrong <i>in our favour</i> and changes on evidence
refuses a reused <code>transaction_id</code> with 422	K10 — that a reuse is a rejection, not an idempotent replay, so a retry after a lost response learns nothing
records the real status for each of eight scenario numbers	K12 — the missing half of the status enumeration. These are written to fail loudly with the exact string
the 180-second pending scenario, at a 240-second timeout	Genuine in-flight state against a real provider — the only way the late-payment grace window is ever proved against something other than our own fixture
an unreachable host classifies as <code>not_transmitted</code>	The classifier's table against a real DNS failure rather than a synthesised error code
a refund cannot be exercised	Not "not yet implemented" — there is nothing to call. K1 and K5

ONE THING NO CREDENTIAL WILL UNLOCK HERE

A real **inbound** webhook needs a publicly reachable URL for iPayMoney to call, which this environment does not have. So the one property only a real delivery can settle — which header name actually arrives, and whether its value is the secret or a signature — stays open as **K7** even with sandbox credentials in hand. The domain side is proved against an injected transport in `ipaymoney-replay.test.ts` instead.

Local, at `c4ff2e6`

COMMAND	RESULT	DETAIL
pnpm lint	CLEAN	5 packages, zero warnings
pnpm typecheck	CLEAN	5 packages, <code>tsc --noEmit</code> under <code>exactOptionalPropertyTypes</code>
pnpm test	478 PASSED	25 files · 478 passed, 16 skipped (494 total)
pnpm build	COMPILED	Production build of the web app
playwright test	31 PASSED	31 tests, 1 worker, against a production build
reconcile-check	CLEAN	drift 0 · 0 unbalanced transactions · net XOF 0 over 8 accounts

Which tests actually ran, and which did not

SUITE	RAN	SKIPPED	WHY SKIPPED
<code>ipaymoney.test.ts</code>	44	0	—
<code>ipaymoney-replay.test.ts</code>	5	0	—
<code>ipaymoney.integration.test.ts</code>	1	16	No sandbox credentials. The one that ran asserts the gate itself, so the skip is visible in the output
Everything else in <code>@afrinext/core</code>	428	0	—

Those 16 are the entire skipped count of the whole workspace suite. Nothing else in Afrinext is skipped, and no existing test was weakened or deleted to make this milestone green.

Continuous integration

RUN	COMMIT	LINT · TYPECHECK · TEST	DEPENDENCY SCAN	BROWSER
#37 · 32464698346	76d664a	SUCCESS	SUCCESS	SUCCESS
#36 · 32464694392	76d664a	SUCCESS	SUCCESS	FAILURE
#38 · 32468173831	936cfc3	SUCCESS	SUCCESS	SUCCESS
#39 · 32468177057	936cfc3	SUCCESS	SUCCESS	SUCCESS
#40 · 32470716170	c4ff2e6	SUCCESS	SUCCESS	SUCCESS

RUN #36 FAILED, AND IT WAS A REAL DEFECT – IN A TEST, NOT IN THE PRODUCT

One browser test failed: `delivery.spec.ts` › *the whole journey, then an unauthorized buyer tries everything*. The webhook it posts answered **400** instead of 200. The same commit passed in run #37, which is exactly the shape of a race.

The cause: the test's `buy()` helper clicked **pay** and then waited for `pay-waiting` before reading the charge's `provider_ref` out of the database. `pay-waiting` is **static text inside the form** — it is on screen before the server action has done anything. So the read raced the charge being created, and when it lost, the webhook was posted naming an empty reference, which the route correctly refused.

This exact bug was found and fixed in `checkout.spec.ts` in an earlier phase, with a comment saying so. `delivery.spec.ts` is a second copy of the same journey and kept the old wait. It has now been corrected the same way — wait for `payment-status` to read `pending`, which appears only once the action has actually completed — and the helper now asserts the reference is non-empty, so a future race fails naming its cause instead of surfacing three lines later as an unexplained 400.

No product code changed. The application behaved correctly throughout: it refused a webhook that named no payment. Only the test was wrong, and only about when to look.

Reconciliation

`reconcile-check.ts` runs after the suite against the CI database and asserts the ledger reconciles: `sum(direction × amount_minor)` against the `account_balances` cache, plus the ten commerce detectors. It reported **drift 0, 0 unbalanced transactions** and a net of **0 XOF** across 8 accounts, over a real capture → settlement → payout cycle. Both self-proving probes behaved: the commerce detector stayed silent on consistent data and raised `succeeded_payment_orphaned` when the gate broke it deliberately, and the refund detector stayed silent and raised `refund_amount_mismatch`. A detector that cannot be shown to fire is not evidence of anything, so the gate breaks its own invariants inside a rolled-back transaction on every run.

Mutation results — 24 of 26 killed

The Phase 3 mutation matrix was re-run in full at `936cfc3`, which is byte-identical to `c4ff2e6` for every file the matrix touches. Each mutation applies a deliberate defect, runs the entire `@afrinext/core` suite, and restores the file from a pristine snapshot.

24 KILLED

2
SURVIVE

Both survivors are the same invariant, defended three times over. R26 removes all three at once and is killed.

The rule the correction added

#	DEFECT INTRODUCED	RESULT	FAILED
R1	A timeout after transmission becomes a definite failure	KILLED	23
R2	<code>ECONNRESET</code> , <code>ETIMEDOUT</code> , <code>EPIPE</code> , <code>ABORT_ERR</code> added to the "proven not transmitted" set	KILLED	4
R3	A provider failure with no <code>notExecutedEvidence</code> is believed	KILLED	2
R4	<code>in_doubt</code> → <code>in_flight</code> made legal, so an unknown could be retried	KILLED	4
R20	A <code>succeeded</code> refund is accepted with no provider reference	KILLED	1

Ordering, idempotency and double-refund defence

#	DEFECT INTRODUCED	RESULT	FAILED
R5	The attempt row is written <i>after</i> the provider call instead of before	KILLED	42
R6	The open-attempt check is removed	KILLED	1
R7	<code>for update of r</code> removed — the row lock	SURVIVES	0
R8	The guarded <code>where status = ?</code> removed from the transition	SURVIVES	0
R26	All three at once — the transition assertion, the row lock and the guarded update	KILLED	3
R9	The derived idempotency key drops the attempt number, so two attempts collide	KILLED	5

WHY R7 AND R8 SURVIVE, AND WHY THAT IS A FINDING RATHER THAN A HOLE

Three independent mechanisms serialise a refund dispatch: `assertRefundTransition` in application code, `SELECT ... FOR UPDATE` on the refund row, and the guarded `UPDATE ... WHERE status = ?`. Any **one** of them is sufficient to stop a double dispatch, so removing any **one** of them leaves behaviour unchanged and every test still passes.

That is what defence in depth *means*, and a surviving mutant here is the evidence that the depth is real rather than nominal. R26 exists to prove the tests are not simply blind: with all three removed, three tests fail. A reviewer should read R7 and R8 as "redundant, deliberately", and R26 as the assertion that the redundancy is the only reason they survive.

Authorization, elevation and the operator surface

#	DEFECT INTRODUCED	RESULT	FAILED
R10	The refund amount is no longer checked against the payment	KILLED	1
R11	A refund may be dispatched for a payment that never succeeded	KILLED	1
R12	<code>authorize()</code> removed from the execution path	KILLED	3
R13	Step-up elevation no longer required	KILLED	1
R19	An operator may confirm their own manual reconciliation	KILLED	1
R21	Secret screening removed before provider text is stored	KILLED	2
R25	The queue's status filter is no longer validated	KILLED	1

Scheduling, clamps and configuration

#	DEFECT INTRODUCED	RESULT	FAILED
R14	The scheduler picks up <code>owed</code> refunds directly, bypassing the claim	KILLED	2
R15	The stuck-refund sweep resolves to <code>failed</code> instead of <code>in_doubt</code>	KILLED	1
R16	The attempt-ceiling clamp is removed, so configuration may raise it without bound	KILLED	1
R17	The backoff clamp is removed	KILLED	1
R18	A malformed setting is coerced rather than refused	KILLED	1

Reconciliation detectors

#	DEFECT INTRODUCED	RESULT	FAILED
R22	The orphaned-refund detector is disabled	KILLED	1
R23	The refund amount-mismatch detector is disabled	KILLED	1
R24	The single-operator detector is disabled	KILLED	1

THE HARNESS ITSELF WAS HARDENED DURING THIS MILESTONE

A mutation escaped into a commit earlier in Phase 3: a killed run left the source mutated, and the next run captured the mutated file as its own backup. It was caught by reading a diff, not by a test.

The matrix now runs in a **detached git worktree**, not in the checkout, restores from a pristine snapshot keyed to that worktree, refuses to start against an already-mutated tree, and holds an `flock` so a second matrix cannot race the first. Both failure modes recurred while running this matrix — a killed run left R1 applied, and two matrices ran concurrently and recorded R1 twice with different counts — and both were caught by the new guards rather than by luck. The contaminated results were discarded and the matrix re-run clean from zero.

Questions for iPayMoney — written, ordered, not sent

Status: NOT SENT. `docs/providers/ipaymoney/support-questions.md` holds the complete message, in French, addressed to `support@i-pay.money`, ready to send. Sending it is a decision for you, not for this milestone — it goes out under AFRI NEXT TECHNOLOGIE's name.

Eighteen questions, ordered by what each answer blocks. K1 and K5 lead because they decide whether Afrinext can return money to a buyer at all, and no amount of engineering changes that.

#	QUESTION	BLOCKS	ANSWER
K1	Is there a customer-refund API? If not, please confirm that explicitly	Everything below it	AWAITING
K5	If there is no API, what is the official procedure for a merchant to return a customer's money — and is that refund then visible in the API or the reports?	Refunding a buyer at all	AWAITING
K2	If a refund API exists, can its status be queried afterwards, and by which identifier?	Resolving an <code>in_doubt</code> refund mechanically	AWAITING
K3	If a refund API exists, does it accept an idempotency key, and does a repeat return the original outcome?	Retrying safely after a lost response	AWAITING
K4	If a refund API exists, which error codes mean — in iPayMoney's words — that no refund was created?	Failure classification	AWAITING
K6	If a refund API exists, does it cover both mobile money and cards, and is there a time limit?	Whether an old <code>refund_due</code> is refundable	AWAITING
K7	Exact header name; is the value the secret transmitted as-is, or a signature computed over the body?	Trusting a webhook with money	AWAITING
K8	Is the transaction amount available anywhere — an undocumented field, another endpoint, a report?	The amount cross-check · Settlement S3	AWAITING
K9	Do webhooks carry a unique event identifier?	Replay protection	AWAITING
K10	Can a payment be found from our own <code>transaction_id</code> when the POST response is lost?	Recovering a lost charge	AWAITING
K11	Is the 3 % collection fee returned when a payment is refunded or cancelled?	Settlement S4	AWAITING
K12	The complete list of possible <code>status</code> values	Reconciliation · status mapping	AWAITING
K13	For XOF, is <code>amount</code> in whole francs — is 15 000 FCFA sent as <code>"15000"</code> ?	Every amount we send	AWAITING
K14	What does a 5xx mean for the transaction — not created, or indeterminate?	Failure classification	AWAITING
K15	Are there rate limits on either endpoint?	Integration	AWAITING
K16	How long before a collected payment is available on the merchant balance?	Settlement S2	AWAITING
K17	Is <code>country</code> the paying customer's country or the merchant's?	Every charge we send	AWAITING
K18	For an Entreprise Individuelle, which Live-activation documents apply and whose identity document is wanted?	Go-live	AWAITING

K17 and K18 were added during this milestone: K17 because `createCharge` cannot be written without knowing whose country to send, and K18 because an Entreprise Individuelle does not fit the documented list cleanly. K2, K3, K4 and K6 were defined in the gate report but had not been carried into the message itself — they are conditional on K1 being answered "yes", and they now travel in the same

envelope rather than in a second round trip. `getRefund()`'s refusal message cites K2, so that citation now resolves to a question somebody can actually send.

TWO OF THESE COST MONEY IF ANSWERED LATE

K13 is a factor of 100 on real money if our reading of XOF's zero decimals is wrong. **K17** is on every single charge. Both are cheap to ask and expensive to discover.

`docs/providers/ipaymoney/live-activation-checklist.md` assembles the documented requirements (L45–L61) so the work can start when somebody decides to start it. **Nothing has been submitted and nothing has been enabled.**

It exists now because Live activation is a **queue with an unknown wait**, not a task: the documentation says the answer arrives by email after examination and gives no service level. Beginning it early costs nothing; discovering it late costs whatever the review takes.

REQUIRED	AFRINEXT'S VALUE	STATE
Nom de l'entreprise	AFRI NEXT TECHNOLOGIE	KNOWN
Secteur d'activité	Commerce et formation en ligne	KNOWN
Téléphone · adresse du siège · site web	—	YOU
N° pièce d'identité · N° registre de commerce · IFU/NIF	—	YOU
Documents: RCCM · IFU/NIF · pièce d'identité	—	YOU

The blank rows are deliberate. Those are business and personal identifiers; they belong in the iPayMoney dashboard, not in a git repository. The checklist is a checklist, not a store.

Two things worth settling before submitting, both in the file:

- **"Site web / Application" means iPayMoney will look at Afrinext.** A person reviews the submission and will open whatever URL is given. Whether the platform is presentable at that moment is a scheduling decision rather than a paperwork one.
- **An Entreprise Individuelle does not fit the documented list.** It asks for both a *registre de commerce* and a *pièce d'identité*, which for an EI are the same person's paperwork. That is K18 — better answered before a rejection than after.

Nothing in the code enables Live. `IPAYMONEY_ENVIRONMENT` has no default precisely so that reaching Live requires somebody to say `live` out loud.

Settlement blockers

Settlement is not started, and this milestone does not start it. Nothing here touched `ledger/flows.ts`, `ledger/post.ts`, settlement holds or payout logic. The list below is what would block Settlement if it were authorised — carried forward from the gate report and updated for what this milestone changed.

#	BLOCKER	MOVED?
S1	Phase 3 has not been approved	UNCHANGED
S2	Settlement timing (T+N) is undocumented — K16. Posting a seller's claim before Afrinext holds the funds is a promise the balance cannot keep	UNCHANGED
S3	The amount is not verifiable from the provider — K8. Booking a capture against a number no provider will confirm makes the ledger authoritative about something only we assert	NOW PROVEN IN CODE — <code>statesChargeAmount === false</code> is asserted by a test rather than noted in a document
S4	The 3 % fee's treatment on refund is unknown — K11. If a refund does not return it, the reversal is not the mirror of the capture	UNCHANGED
S5	There is no refund execution to reverse — K1, K5	UNCHANGED
S6	No real payment has ever been made. Every capture in every test came from the mock or an injected transport	UNCHANGED — no sandbox credentials
S7	New. <code>initiatePayment</code> does not pass the buyer's phone or country, so no iPayMoney charge can be created through the checkout path at all	FOUND BY THIS MILESTONE

S2, S3, S4 and S5 are answered by iPayMoney, not by us. S6 needs credentials. S7 is ours and is small. That distribution is the case for sending the questions before building anything else.

Decisions needed from you

Three, and only the first is urgent.

1 Send the questions to iPayMoney

The message in `support-questions.md` is written and waiting. It goes out under AFRI NEXT TECHNOLOGIE's name, so it is yours to send, not mine. **K1** and **K5** decide whether Afrinext can ever return a buyer's money automatically; **K13** and **K17** affect every charge. Everything else in this integration is blocked behind answers only iPayMoney has.

If you would rather send a shorter message first, **K1**, **K5**, **K7**, **K13** and **K17** are the five that block work today.

2 Provide sandbox credentials, or confirm there are none yet

Sixteen tests are written and skipped. With `IPAYMONEY_BASE_URL`, `IPAYMONEY_API_KEY` and `IPAYMONEY_ENVIRONMENT=sandbox` in the environment they run, and the status enumeration (**K12**) and the amount question (**K8**) get answered by machine rather than by email. Without them, the adapter stays a hypothesis.

The key is a secret. It goes in the environment — never in this repository and never in a CI log.

3 Whether to start the Live activation submission now

It is a queue with no published service level, and the information it needs is yours. Starting it does not commit anything: the code cannot reach Live without somebody setting `IPAYMONEY_ENVIRONMENT=live` deliberately. Worth answering **K18** first if the EI paperwork is uncertain.

NO DECISION WAS TAKEN THAT CHANGES AN APPROVED RULE

The refund state machine, the `in_doubt` rules, the attempt-before-dispatch ordering, the late-payment policy and the dual-control reconciliation are all exactly as approved. The webhook choice was put to you before it was implemented, and you chose A. Nothing in this milestone needs retroactive approval — these three are about what happens next.

What was NOT done, on purpose

- **Settlement is not started.** No ledger, payout or settlement file was touched.
- **Nothing is merged to `main`.** All work is on `claude/create-app-repository-1cxsih`.
- **No production readiness is claimed.** The adapter has never spoken to iPayMoney.
- **No refund functionality was invented.** `refund()` and `getRefund()` throw, and will keep throwing until iPayMoney confirms a real customer-refund capability.

- **No payment verification was weakened** to compensate for the documentation being incomplete. The webhook path became *stricter*, not looser: it now re-reads the truth from the provider instead of accepting what it is told.